ABSTRACT

The rapid evolution of financial markets has led to an exponential increase in the volume and velocity of market data. Modern trading systems require extremely low latency and deterministic execution to respond effectively to dynamic price movements. Traditional software-based trading systems, although flexible, suffer from limitations such as operating system overhead, unpredictable latency, and limited parallel processing capabilities.

This project presents the design and simulation of a hardware-based stock trading signal generator implemented using Field Programmable Gate Arrays (FPGAs). The proposed system processes stock price and volume data in real time and generates trading signals in the form of BUY, SELL, or HOLD decisions. The design leverages widely used financial indicators such as Exponential Moving Average (EMA), Relative Strength Index (RSI), and volume spike detection to analyze market trends.

The system is implemented at the Register Transfer Level (RTL) using Verilog Hardware Description Language. Fixed-point arithmetic (Q16.16 format) is used to efficiently represent decimal values without the overhead of floating-point computation. The architecture is fully synchronous and pipeline-based, enabling parallel execution of multiple modules such as EMA calculators, RSI computation unit, and volume filter.

Simulation results demonstrate that the system correctly identifies trend reversals using EMA crossover logic and generates corresponding trading signals. The hardware-based approach ensures deterministic timing and significantly lower latency compared to software implementations.

This project highlights the potential of FPGA-based systems in high-frequency trading applications and serves as a foundational step toward building real-time, low-latency financial analytics systems.

CHAPTER 1: INTRODUCTION

1.1 Background

Financial markets have undergone a massive transformation over the past few decades, shifting from manual trading floors to highly automated electronic systems. Earlier, trading decisions were made by human traders based on experience, intuition, and delayed market data. However, with advancements in technology, modern financial systems now rely heavily on algorithmic trading, where decisions are made using predefined mathematical models and executed automatically.

One of the most advanced forms of algorithmic trading is **High-Frequency Trading (HFT)**, where trading systems operate at extremely high speeds, executing thousands to millions of trades within fractions of a second. In such environments, the ability to process data quickly and respond instantly to market changes becomes critically important.

1.2 Need for Real-Time Trading Systems

In financial markets, prices fluctuate continuously based on supply and demand. Traders aim to exploit small price differences to generate profit. However, these opportunities exist only for very short durations, often lasting just microseconds.[6], [10]

Therefore, modern trading systems must:

- Process incoming market data in real-time
- Analyze trends instantly
- Generate trading signals without delay
- Execute trades at extremely high speed

Even a small delay in processing can lead to missed opportunities or financial losses. This has created a strong demand for ultra-low latency systems capable of handling real-time data efficiently.[5], [11]

1.3 Limitations of Conventional Software-Based Systems

Most traditional trading systems are implemented using software running on CPUs. While CPUs are flexible and easy to program, they suffer from several limitations when used in high-speed trading applications:

1.3.1 Latency Overhead

Software systems involve multiple layers such as operating systems, memory access, and instruction pipelines. These introduce delays that are difficult to eliminate completely.

1.3.2 Sequential Processing

Although modern CPUs support multi-threading, they still execute instructions sequentially at the core level, limiting true parallel processing.

1.3.3 Non-Deterministic Behavior

Execution time in software systems is not always predictable due to interrupts, context switching, and caching effects.

1.3.4 Limited Throughput

Handling massive streams of real-time data efficiently is challenging for software-based systems. Due to these limitations, software-based trading systems struggle to meet the performance requirements of high-frequency trading environments.

1.4 Introduction to Hardware Acceleration

To overcome the limitations of software systems, hardware acceleration techniques are increasingly being used. Hardware acceleration involves implementing critical algorithms directly in hardware instead of software, allowing faster and more efficient execution. One of the most promising technologies for this purpose is the **Field Programmable Gate Array (FPGA)**.[8]

1.5 Field Programmable Gate Arrays (FPGAs)

An FPGA is a reconfigurable hardware device that can be programmed to implement custom digital circuits. Unlike CPUs, which execute instructions, FPGAs execute operations directly in hardware, enabling true parallelism.

Key Features of FPGA:

- **Parallel Processing:** Multiple operations can be executed simultaneously
- **Low Latency:** Eliminates software overhead
- **Deterministic Performance:** Predictable execution timing
- **Custom Architecture:** Can be optimized for specific applications

Because of these features, FPGAs are widely used in applications requiring high speed and low latency, such as signal processing, networking, and financial trading systems.

1.6 FPGA in Financial Trading

In the context of financial trading, FPGAs are used to accelerate critical operations such as:

- Market data processing
- Technical indicator computation
- Trading signal generation
- Order execution

By implementing these operations in hardware, FPGAs significantly reduce latency and improve overall system performance.[9]

For example, while software-based systems operate in microseconds, FPGA-based systems can achieve execution times in nanoseconds, providing a major competitive advantage.

1.7 Trading Indicators Used in This Project

Trading decisions are often based on mathematical indicators derived from historical price and volume data. This project focuses on three key indicators:

1.7.1 Exponential Moving Average (EMA)

EMA gives more importance to recent prices, making it more responsive to market changes compared to simple averages.[25], [27]

1.7.2 Relative Strength Index (RSI)

RSI measures the strength of price movements and helps identify overbought or oversold conditions.[25], [26]

1.7.3 Volume Analysis

Volume analysis helps detect unusual trading activity, which can indicate potential market movements. These indicators are widely used in financial markets and are suitable for hardware implementation due to their repetitive and parallelizable nature.

1.8 Problem Statement

Despite the advantages of FPGA technology, designing a complete hardware-based trading system is complex and requires a deep understanding of both financial algorithms and digital hardware design.

1.9 Objectives of the Project

The primary objectives of this project are:

- To design a hardware-based trading system using Verilog
- To implement key trading indicators such as EMA and RSI
- To generate trading signals based on crossover logic and market conditions
- To achieve low-latency processing suitable for real-time applications
- To simulate and verify the system using testbench and waveform analysis

1.10 Scope of the Project

This project focuses on:

- Simulation of a trading system using FPGA design principles
- Implementation of mathematical indicators in hardware
- Signal generation logic based on predefined conditions

However, the project does not include:

- Real-time market data integration
- Direct connection to stock exchanges
- Live trading execution

1.11 Proposed System Overview

The proposed system consists of multiple modules working together:

1. **Input Processing Module** – receives price and volume data
2. **Indicator Modules** – computes EMA, RSI, and volume averages
3. **Signal Logic Module** – generates BUY/SELL/HOLD signals
4. **Output Module** – displays signals using LEDs

All modules are implemented in hardware using Verilog and operate synchronously with a clock signal.

1.12 Significance of the Project

This project demonstrates how hardware-based design can be used to solve real-world problems in financial systems. It highlights:

- The importance of low-latency computation
- The advantages of FPGA over traditional systems
- The feasibility of implementing trading algorithms in hardware

It also serves as a foundation for more advanced systems in high-frequency trading.

1.13 Organization of the Report

This report is structured to gradually build an understanding of the proposed FPGA-based stock signal generation system, starting from background concepts and progressing toward implementation, results, and analysis.

Chapter 2: Literature Review
This chapter discusses previous research and existing solutions related to high-frequency trading and FPGA-based systems. It examines how financial algorithms such as EMA and RSI have been

used in trading and highlights the limitations of software-based approaches. The insights from this chapter help in identifying the motivation and direction for the proposed work.

Chapter 3: System Overview

This chapter provides a high-level description of the overall system. It explains how different components are connected and how data flows through the system. The block diagram and module interaction are introduced here to give a clear conceptual understanding before diving into detailed design.

Chapter 4: Methodology and Design Approach

In this chapter, the internal working of the system is explained in detail. It covers the design of key modules such as EMA calculation, RSI computation, volume filtering, and signal generation logic. The use of fixed-point representation and hardware-oriented design techniques is also discussed to justify design choices.

Chapter 5: Implementation

This chapter focuses on how the design is translated into hardware using Verilog. It describes the structure and functionality of individual modules and explains how they are integrated to form the complete system. Important code-level decisions and design considerations are also presented.

Chapter 6: Simulation and Results

This chapter presents the simulation outcomes of the proposed system. It includes waveform analysis and explains how BUY, SELL, and HOLD signals are generated based on EMA crossover and other conditions. Different test scenarios are discussed to validate system behavior.

Chapter 7: Performance Analysis

This chapter evaluates the system in terms of speed, efficiency, and latency. It also provides a comparison between FPGA-based implementation and traditional software-based systems to highlight the advantages of hardware acceleration.

Chapter 8: Challenges and Limitations

This chapter reflects on the practical difficulties faced during the project, including debugging issues, timing synchronization, and design constraints. It also discusses the current limitations of the system.

Chapter 9: Conclusion and Future Work

The final chapter summarizes the overall work and key findings of the project. It also suggests possible improvements and future enhancements, such as integrating real-time market data and extending the trading strategy.

CHAPTER 2: LITERATURE REVIEW

With the rapid growth of financial markets and the increasing use of automated trading systems, there has been a strong demand for faster and more efficient ways to process market data. Trading strategies that once relied on human decision-making are now implemented using algorithms that can analyze data and respond within fractions of a second.

In such environments, system performance becomes critical. Even a small delay in processing or decision-making can result in missed trading opportunities. This has led researchers and industry professionals to explore alternative computing approaches beyond traditional software-based systems.

One such approach is the use of Field Programmable Gate Arrays (FPGAs), which allow trading algorithms to be implemented directly in hardware. This chapter reviews existing work in the areas of high-frequency trading, FPGA-based systems, and commonly used financial indicators such as EMA and RSI.

2.1 High-Frequency Trading Systems

High-Frequency Trading (HFT) is a form of algorithmic trading that focuses on executing a large number of trades at extremely high speeds. These systems rely on real-time market data and predefined algorithms to make rapid decisions.[6], [11] ,[12]

The key characteristics of HFT systems include:

- Very low latency execution
- High-speed data processing
- Automated decision-making
- Large number of transactions in a short time

In such systems, even microsecond-level delays can significantly impact profitability. As a result, optimizing system performance is one of the primary concerns in trading system design.

2.2 Limitations of Traditional Software-Based Systems

Most conventional trading systems are implemented using software running on CPUs. While these systems offer flexibility and ease of development, they have several limitations when used in high-speed trading environments.

One major limitation is latency introduced by the operating system. Tasks such as scheduling, memory access, and interrupt handling add delays that cannot be completely eliminated.

Another issue is the lack of true parallelism. Although modern processors support multi-threading, they still execute instructions sequentially at the hardware level. This limits their ability to process large volumes of data simultaneously.

Additionally, software systems often exhibit non-deterministic behavior, meaning execution time can vary depending on system conditions. This unpredictability is undesirable in trading systems where consistent timing is important.

Due to these limitations, software-based approaches are not always suitable for applications that require ultra-low latency and high throughput.

2.3 FPGA-Based Trading Systems

FPGAs provide a hardware-based solution to overcome the limitations of software systems. Unlike CPUs, which execute instructions sequentially, FPGAs allow the creation of custom circuits that can perform multiple operations in parallel.

According to industry resources such as Intel (Altera) financial solutions, FPGAs are widely used in trading systems to accelerate data processing and reduce latency. They enable real-time execution of algorithms without the overhead of an operating system.

Companies like IMC and Optiver have also adopted FPGA-based systems to improve trading performance. These systems process market data, compute indicators, and generate trading signals directly in hardware, resulting in faster and more reliable execution.[9], [21], [22]

2.4 Industry Applications of FPGA in Trading

Several organizations have successfully implemented FPGA-based trading solutions:

- **Intel (Altera):** Provides FPGA platforms specifically designed for financial applications, offering low-latency processing and high-speed networking.
- **IMC Trading:** Uses FPGAs for market data processing and signal generation to achieve faster decision-making.
- **Optiver:** Implements FPGA hardware to improve execution speed and reduce latency in trading strategies.

- **Enyx (Exegy):** Develops FPGA-based infrastructure for high-speed trading systems, focusing on efficient data handling and execution pipelines.
- **Magmio:** Highlights various use cases of FPGA in trading, emphasizing its ability to handle multiple data streams simultaneously and execute logic at very high speeds.

These examples clearly demonstrate that FPGA technology is already widely used in real-world trading systems.

2.5 Technical Indicators in Trading Systems

Trading systems often rely on mathematical indicators derived from price and volume data. Among these, moving averages and momentum indicators are commonly used.

2.5.1 Exponential Moving Average (EMA)

EMA is a type of moving average that gives more weight to recent prices. This makes it more responsive to changes in market conditions compared to simple moving averages.[25], [27]

2.5.2 Relative Strength Index (RSI)

RSI is a momentum indicator that measures the strength of price movements. It is used to identify overbought and oversold conditions in the market.[25], [26]

2.5.3 Volume Analysis

Volume is an important factor in trading, as it indicates the level of market activity. Sudden increases in volume can signal strong price movements.These indicators are suitable for hardware implementation because they involve repetitive calculations that can be efficiently parallelized.

2.6 Comparison Between FPGA and CPU-Based Systems

Table 1: Comparison between FPGA and CPU based Trading System

Parameter	CPU-Based System	FPGA-Based System
Latency	Higher	Very Low
Execution	Sequential	Parallel
Predictability	Variable	Deterministic

Throughput	Limited	High
Flexibility\	High	Moderate

From this comparison, it is clear that FPGA-based systems offer significant advantages in terms of speed and efficiency, making them suitable for high-frequency trading applications.

2.7 Research Gap

Although FPGA-based trading systems provide significant performance improvements, they are often complex and difficult to design. Most existing solutions are developed for industrial use and are not easily accessible for academic learning.

There is a need for a simplified and well-structured FPGA-based trading system that demonstrates the core concepts of hardware acceleration, signal generation, and real-time processing.

2.8 Motivation for the Proposed Work

The main motivation behind this project is to explore how financial algorithms can be implemented in hardware and to demonstrate the advantages of FPGA-based systems in real-time applications.

By designing a system that uses EMA, RSI, and volume-based logic, this project aims to provide a practical understanding of how trading strategies can be translated into hardware and executed efficiently.

2.9 Summary

This chapter reviewed existing research and industry practices related to high-frequency trading and FPGA-based systems. It highlighted the limitations of software-based approaches and demonstrated the advantages of hardware acceleration.

Based on these observations, the proposed system focuses on implementing a simplified FPGA-based trading signal generator that is efficient, scalable, and suitable for academic study.

CHAPTER 3: SYSTEM OVERVIEW

This chapter presents a high-level description of the proposed FPGA-based stock signal generation system. The system is designed to process stock price and volume data in real time and generate trading signals based on predefined conditions.

The overall design follows a modular approach, where each component performs a specific task such as data storage, indicator calculation, or signal generation. These modules work together in a synchronized manner to produce the final output.

3.1 Overall System Architecture

The system consists of the following major components:

- Input Processing Unit
- Shift Registers (Data Storage)
- Indicator Calculation Units (EMA, RSI)
- Volume Analysis Unit
- Signal Generation Logic
- Output Display (LEDs / Signals)

3.2 Working Principle

The system operates in a sequential manner driven by a clock signal. At every clock cycle, a new price and volume sample is provided as input.

The key steps involved are:

1. The input price is stored in shift registers to maintain historical data.
2. EMA modules compute fast and slow moving averages.
3. RSI module calculates momentum based on price changes.
4. Volume filter checks for abnormal trading activity.
5. Signal logic compares all indicators and generates a trading signal.

The entire system works continuously in a pipeline manner, ensuring real-time processing.

3.3 Block Diagram

Figure 1: Block Diagram of pipeline

3.4 Data Flow Description

The flow of data in the system can be summarized as follows:

- Input data enters the system through price_in and volume_in.
- The data is passed through shift registers to maintain a history of values.
- Indicator modules process this data in parallel.
- The outputs of all modules are fed into the signal logic unit.
- The signal logic generates the final trading decision.

3.5 Flowchart of System Operation

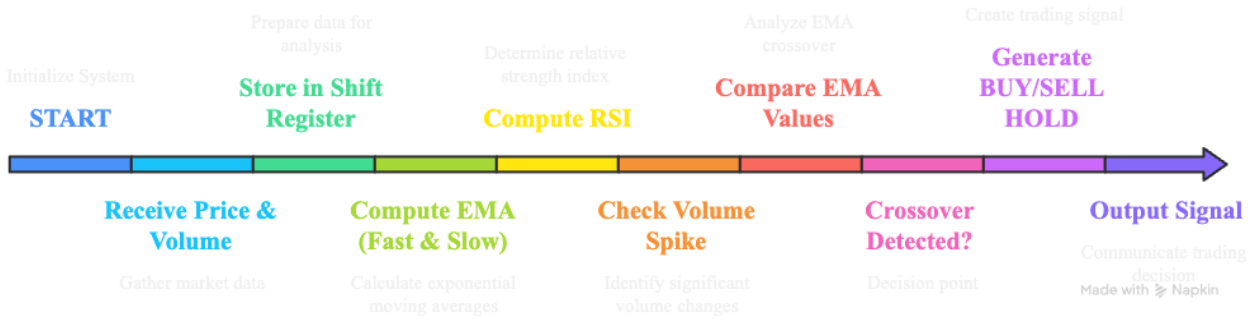


Figure 2: Flowchart of FPGA-Based Trading Signal Generation System

3.6 Module Description (High-Level)

3.6.1 Shift Register

Stores previous price values and provides required data for calculations like SMA and RSI.

3.6.2 EMA Module

Calculates exponential moving average using current price and previous EMA value.

- EMA-14 → fast response
- EMA-50 → smooth response

3.6.3 RSI Module

Calculates market momentum based on gains and losses in price.

3.6.4 Volume Filter

Detects sudden increase in trading volume, indicating strong market activity.

3.6.5 Signal Logic

This is the decision-making unit. It checks:

- EMA crossover
- RSI conditions
- Volume spike

and generates: **BUY / SELL / HOLD**

3.7 Input and Output Description

Inputs:

- price_in → stock price (Q16.16 format)
- volume_in → trading volume
- clk → clock signal
- rst → reset

Outputs:

- signal_out → trading signal
    - 01 → BUY
    - 10 → SELL
    - 00 → HOLD
- signal_valid → indicates valid output
- led → visual indication

3.8 Design Approach

The system is designed using a hardware-oriented approach:

- Fully synchronous design
- Modular architecture
- Parallel execution of indicators

- Fixed-point arithmetic for efficiency

This approach ensures high performance and low latency.

3.9 Summary

This chapter presented the overall architecture and working of the proposed system. It explained how different modules interact and how data flows through the system to generate trading signals. The next chapter will provide a detailed explanation of the design methodology and implementation of each module.

CHAPTER 4: METHODOLOGY AND DESIGN APPROACH

This chapter explains the design methodology adopted for implementing the FPGA-based stock signal generation system. The system is designed using a modular and hardware-oriented approach, where each component performs a specific function such as data storage, indicator computation, and signal generation.

The design focuses on achieving low latency, parallel execution, and efficient resource utilization by using fixed-point arithmetic and synchronous digital design principles.

4.1 Design Approach

The system is implemented using a **Register Transfer Level (RTL)** design methodology in Verilog. Each module is designed to operate synchronously with a clock signal, ensuring predictable and stable behavior.

Key design principles used in this project include:

- **Modular Design:** Each functionality (EMA, RSI, Volume, Signal Logic) is implemented as a separate module.
- **Parallel Processing:** Indicator calculations are performed simultaneously.
- **Pipeline Architecture:** Data flows through different stages in a sequential manner.
- **Fixed-Point Arithmetic:** Used instead of floating-point to reduce hardware complexity.

4.2 Fixed-Point Representation (Q16.16 Format)

In FPGA design, floating-point arithmetic is resource-intensive. Therefore, this system uses **Q16.16 fixed-point format**, where:

- Upper 16 bits → integer part
- Lower 16 bits → fractional part

Example:
142.75 → 142.75 × 65536 = 9355264

Advantages:

- Efficient hardware implementation
- Faster computation
- Reduced resource usage

4.3 Shift Register Design

Shift registers are used to store historical values of both price and volume data. These values are required for calculating moving averages and other indicators.

Working:

- At every clock cycle, new data is inserted
- Existing data shifts by one position
- Oldest value is discarded

Purpose:

- Maintain past data samples
- Enable rolling computations

4.4 Exponential Moving Average (EMA)

4.4.1 Concept

EMA is used to smooth price data and identify trends. It gives more importance to recent values, making it more responsive than simple averages.

4.4.2 Formula

The Exponential Moving Average (EMA) is calculated as:

$$EMA = \alpha \times Price_{current} + (1 - \alpha) \times EMA_{previous}$$

Where:

- EMA: Exponential Moving Average
- Price_current: Current price value
- EMA_previous: Previous EMA value
- α (alpha): Smoothing factor, calculated as $\alpha = 2 / (N + 1)$

- N: Number of periods

EMA gives more weight to recent prices, making it more responsive to market changes compared to simple moving averages.

4.4.3 Alpha Calculation

The smoothing factor (α) used in Exponential Moving Average (EMA) is calculated as:

$\alpha = 2 / (N + 1)$

Where:

- α (alpha): Smoothing factor
- N: Number of periods

The value of α determines how much weight is given to recent prices. A higher α makes the EMA more responsive to recent changes.

4.4.4 EMA Variants Used

Table 2: EMA Comparison

Type	Period(N)	Alpha
Fast EMA	14	0.133
Slow EMA	50	0.039

4.4.5 Hardware Implementation

- Multiply price with alpha
- Multiply previous EMA with (1 - alpha)
- Add both values
- Shift result to maintain Q16.16 format

Key Features:

- Single clock cycle latency
- Continuous update
- Pipeline-friendly

4.5 Relative Strength Index (RSI)

4.5.1 Concept

RSI is a momentum indicator used to measure the strength of price movements.

- RSI > 70 → Overbought
- RSI < 30 → Oversold

4.5.2 Calculation Steps

1. Calculate price difference
2. Separate gain and loss
3. Compute average gain and loss
4. Calculate RSI

4.5.3 Formula

The Relative Strength Index (RSI) is calculated as:

RSI = 100 × (Avg Gain / (Avg Gain + Avg Loss))

Where:

- RSI: Relative Strength Index (ranges from 0 to 100)
- Avg Gain: Average of positive price changes over a given period
- Avg Loss: Average of negative price changes over the same period

4.5.4 Hardware Considerations

- Uses fixed-point arithmetic
- Requires division operation
- Implemented using sequential logic

4.6 Volume Analysis

Volume plays an important role in confirming market movements.

4.6.1 Objective

To detect abnormal trading activity using volume spikes.

4.6.2 Method

- Calculate average volume over 20 samples
- Compare current volume with threshold

Condition: Volume_current > 1.5 × Average Volume

4.6.3 Implementation

- Shift register stores past volume values
- Running sum is maintained
- Average is computed using reciprocal multiplication

4.7 Signal Generation Logic

This is the most critical part of the system, where all computed indicators are combined to generate trading signals.

4.7.1 Crossover Detection

A crossover occurs when:

- Fast EMA crosses above Slow EMA → BUY
- Fast EMA crosses below Slow EMA → SELL

4.7.2 Detection Method

The system stores the previous comparison result and compares it with the current state.

Previous: fast < slow
Current: fast > slow
BUY signal

4.7.3 Signal Conditions

Table 3: Trading Signal Generation Conditions

Condition	Output
Fast crosses above Slow	Buy
Fast crosses below Slow	Sell
Otherwise	Hold

4.7.4 Implementation Strategy

- Register previous state
- Compare with current state
- Generate output accordingly

4.8 Pipeline and Parallel Processing

The system is designed using a pipeline architecture where:

- Data flows through multiple stages
- Each stage performs a specific operation
- Multiple operations run simultaneously

Benefits:

- Reduced latency
- Increased throughput
- Efficient hardware utilization

4.9 Design Trade-offs

While designing the system, certain trade-offs were considered:

Table 4: Design Trade-offs and Implementation Decisions

Factor	Decision
Precision vs Speed	Fixed-point used
Complexity vs Performance	Modular design
Flexibility vs Efficiency	Hardware optimized

4.10 Summary

This chapter described the methodology used to design the FPGA-based trading system. It explained the implementation of key components such as EMA, RSI, and volume filtering, along with the signal generation logic. The use of fixed-point arithmetic, pipeline architecture, and parallel processing ensures that the system operates efficiently with low latency. The next chapter will focus on the implementation details and code-level explanation of the system.

CHAPTER 5: IMPLEMENTATION

This chapter describes the practical implementation of the proposed FPGA-based stock signal generation system using Verilog Hardware Description Language (HDL). The design is developed at the Register Transfer Level (RTL) and follows a modular approach, where each component is implemented as an independent module and later integrated into a top-level system.

The implementation focuses on achieving correct functionality, efficient data flow, and compatibility with FPGA synthesis tools such as Xilinx Vivado.

5.1 Design Environment and Tools

The system is developed and simulated using the following tools:

Table 5: Design and Simulation Tools Used

Tool	Purpose
Xilinx Vivado	Design synthesis and simulation
Verilog HDL	Hardware description
XSim / GTKWave	Waveform analysis

5.2 Top-Level Module Description

The entire system is integrated in a top module named: **stock_signal_top.v**

Functionality:

- Connects all submodules
- Routes input signals to processing units
- Combines outputs to generate final signal

Inputs:

- clk → system clock
- rst → reset signal
- en → enable signal
- price_in → input price (Q16.16)
- volume_in → input volume

Outputs:

- signal_out → BUY/SELL/HOLD
- signal_valid → validity flag
- led → indicator output

5.3 Shift Register Implementation

Module Name: **shift_reg.v**

Purpose: Stores historical values of price and volume for moving calculations.

Working:

- On each clock cycle, new data is inserted
- Existing values shift forward
- Oldest value is removed

Key Feature:

- Parameterized depth (14, 50, etc.)
- Flattened output for easy access

5.4 EMA Module Implementation

Module Name: **ema.v**

Function: Calculates exponential moving average using fixed-point arithmetic.[27]

Implementation Steps:

1. Multiply current price with alpha
2. Multiply previous EMA with (1 - alpha)
3. Add both results

Code Concept:

```
ema_out <= (ALPHA * price_in >> 16) +
        (ONE_M_ALPHA * ema_out >> 16);
```

Key Points:

- Uses 64-bit intermediate variables
- Outputs updated every clock cycle
- Valid signal ensures correct timing

5.5 SMA Module Implementation

Module Name: **sma.v**

Purpose: Computes simple moving average using running sum.

Working:

- Add new sample
- Subtract oldest sample
- Multiply with reciprocal

Advantage:

- Avoids division using multiplication

5.6 RSI Module Implementation

Module Name: **rsi_calc.v**

Function: Calculates RSI based on price changes.[26]

Key Steps:

- Compute price difference
- Separate gain and loss
- Apply smoothing
- Compute RSI

Implementation Note:

- Division handled using integer arithmetic
- Uses sequential logic

5.7 Volume Filter Implementation

Module Name: **volume_filter.v**

Purpose: Detects volume spikes.

Working:

- Maintains running sum of last 20 samples
- Computes average volume
- Compares with threshold (1.5×)

Condition:

vol_spike = (current_volume > 1.5 × avg_volume)

5.8 Signal Logic Implementation

Module Name: **signal_logic.v**

Function: Generates BUY, SELL, or HOLD signals.

Core Logic:

- Compare EMA14 and EMA50
- Detect crossover using previous state

Code Concept:

```
if (fast_above && !fast_above_d)
  signal_out <= BUY;

else if (!fast_above && fast_above_d)
  signal_out <= SELL;
```

Important Design Feature:

- Uses registered comparison
- Avoids glitches
- Ensures correct edge detection

5.9 Testbench Implementation

Module Name: **tb_stock_signal.v**

Purpose: Simulates system behavior under different conditions.

Key Features:

- Generates clock signal
- Applies test inputs
- Creates trends (up/down)
- Prints signal output

Test Strategy:

- Downtrend → expect HOLD
- Uptrend → expect BUY
- Downtrend → expect SELL

5.10 Integration of Modules

All modules are connected in the top module:

- EMA outputs → signal logic
- RSI output → signal logic
- Volume output → signal logic

The system works synchronously, with all modules updating on the same clock edge.

5.11 Design Considerations

During implementation, the following were considered:

- Avoiding overflow using 64-bit registers
- Maintaining precision using fixed-point format
- Ensuring proper timing using valid signals
- Keeping design modular and scalable

5.12 Summary

This chapter explained the implementation of the FPGA-based trading system using Verilog. Each module was described in terms of functionality and design approach.

The next chapter presents simulation results and validates the system behavior using waveform analysis.

CHAPTER 6: SIMULATION AND RESULTS

This chapter presents the simulation results of the FPGA-based stock signal generation system. The objective of simulation is to verify the correctness of the design and ensure that the system generates appropriate BUY, SELL, and HOLD signals based on input price trends. The simulation is carried out using the Vivado simulator (XSim), and the results are analyzed through waveform outputs and console logs.

6.1 Simulation Setup

The simulation is performed using a testbench designed to mimic real market behavior. The testbench generates a sequence of price and volume inputs representing different market conditions.

Key Simulation Parameters:

Table 6: Different Parameters Used

Parameter	value
Clock Period	10ns
Data Format	Q 16.11
Input Type	Integer scaled to fixed-point
Simulation Tool	Vivado XSim

6.2 Testbench Description

The testbench is responsible for:

- Generating clock signal
- Providing reset and enable signals
- Feeding price and volume data
- Creating artificial trends (uptrend and downtrend)

Input Pattern Used:

The simulation includes three major phases:

1. **Downtrend Phase**
   Price decreases steadily over time
2. **Uptrend Phase**
   Price increases sharply
3. **Downtrend Phase Again**
   Price decreases again to test SELL condition

6.3 Waveform Observation

Figure3: Simulation Output Showing BUY Signal Generation

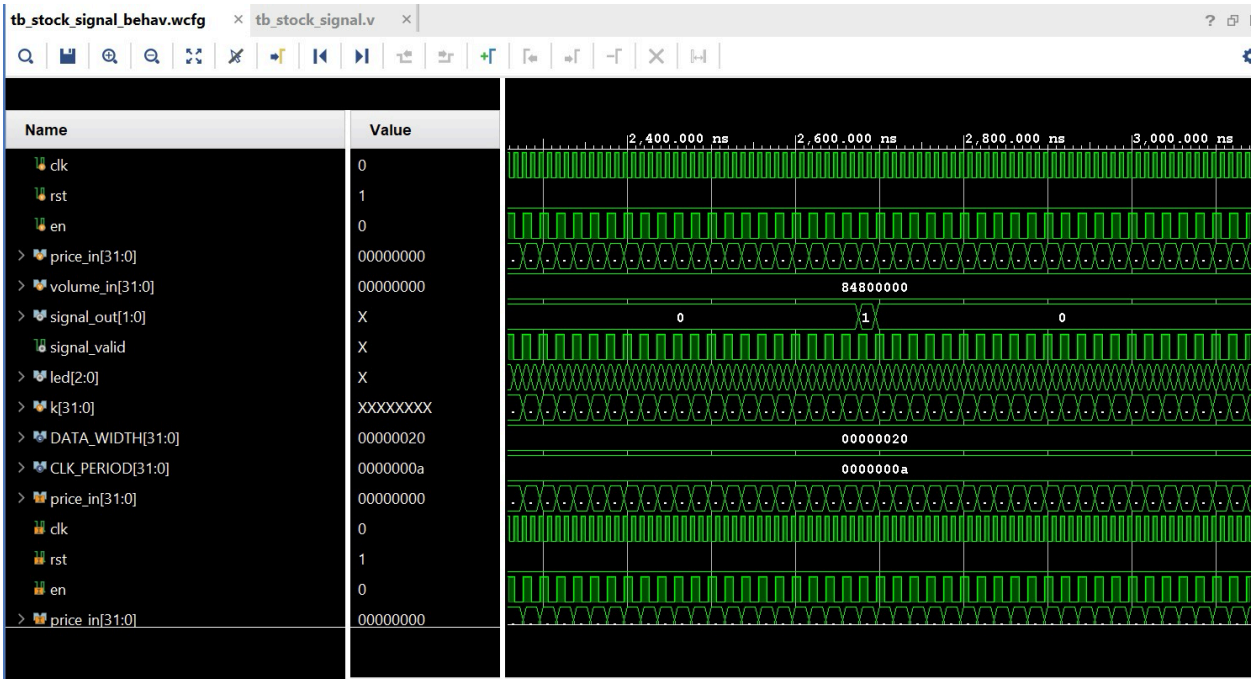
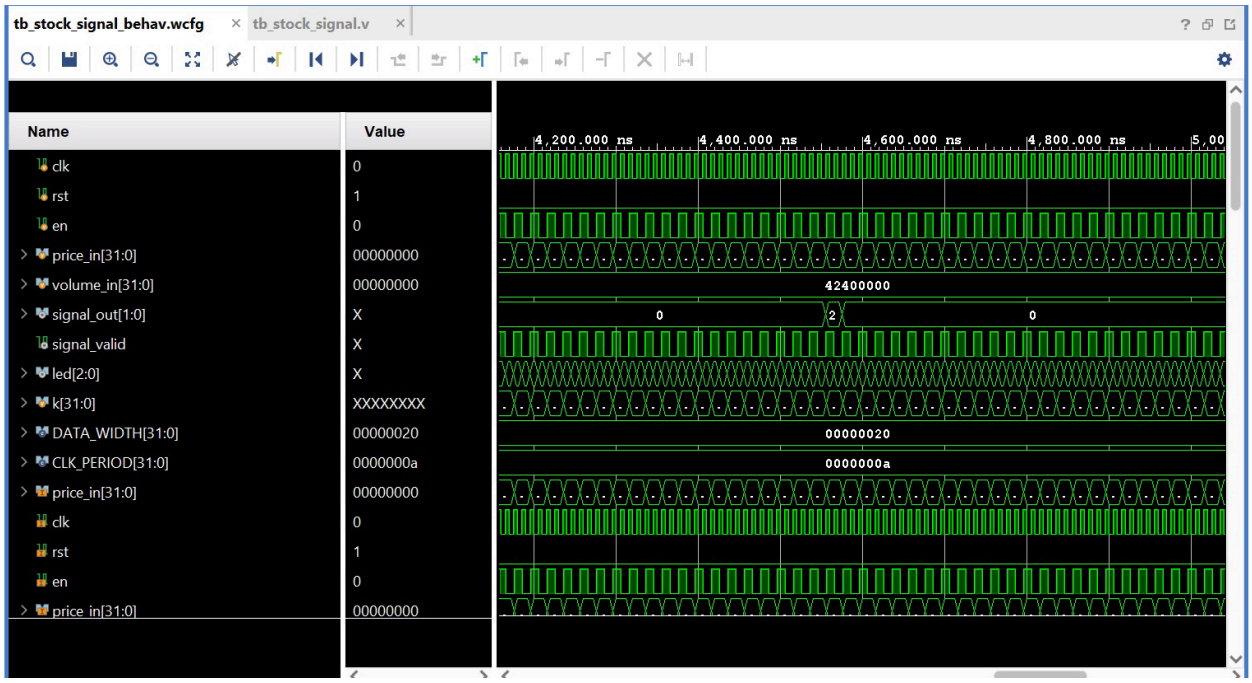


Figure 4: Simulation Output Showing SELL Signal Generation



Observed Signals:

- price_in
- ema14_out (fast EMA)
- ema50_out (slow EMA)
- signal_out
- signal_valid

6.4 EMA Behavior Analysis

The waveform clearly shows the behavior of both EMA signals:

- **EMA-14 (fast EMA)** closely follows the input price
- **EMA-50 (slow EMA)** changes gradually and smooths fluctuations

This confirms that: Fast EMA reacts quickly and Slow EMA reacts slowly

6.5 Crossover Detection

The system successfully detects crossover points:

BUY Signal Generation

- Occurs when EMA-14 crosses above EMA-50
- Indicates upward trend

Condition:
Previous → EMA14 < EMA50
Current → EMA14 > EMA50
BUY

SELL Signal Generation

- Occurs when EMA-14 crosses below EMA-50
- Indicates downward trend

Condition:
Previous → EMA14 > EMA50
Current → EMA14 < EMA50
SELL

6.6 Signal Output Analysis

The output signal is represented as:

Table 7: Different Signal Condition

Condition	Output
01	Buy
10	Sell
00	Hold

Observations:

- During stable regions → HOLD
- During upward crossover → BUY

- During downward crossover → SELL

The signal_valid signal ensures that outputs are considered only when valid.

6.7 Timing Behavior

The system operates synchronously with the clock signal:

- All modules update on the rising edge of the clock
- EMA calculations introduce minimal latency
- Signal generation occurs within 1–2 clock cycles

This demonstrates efficient pipeline behavior.

6.8 Verification of System Functionality

The simulation confirms that:

- EMA modules compute correct values
- Crossover logic works accurately
- Signals are generated at correct points
- System behaves consistently under different input conditions

6.9 Observations and Insights

- Strong trends are required to clearly observe EMA crossover
- Fast EMA responds immediately to price changes
- Slow EMA provides stability and trend confirmation
- Signal generation depends on relative movement, not absolute values

6.10 Limitations in Simulation

- Input data is artificially generated
- No real-time market data used
- Volume and RSI conditions were simplified during testing

6.11 Summary

This chapter demonstrated the simulation and validation of the proposed system. The results confirm that the FPGA-based design correctly generates trading signals based on EMA crossover logic. The next chapter evaluates the performance of the system and compares it with traditional software-based approaches.

CHAPTER 7: PERFORMANCE ANALYSIS

This chapter evaluates the performance of the proposed FPGA-based stock signal generation system. The analysis focuses on key parameters such as latency, throughput, efficiency, and overall system behavior. Additionally, a comparison is made between FPGA-based implementation and traditional software-based systems to highlight the advantages of hardware acceleration.

7.1 Performance Metrics

To evaluate the system effectively, the following performance parameters are considered:

7.1.1 Latency

Latency refers to the time taken by the system to process an input and generate the corresponding output.

7.1.2 Throughput

Throughput indicates the number of data samples processed per unit time.

7.1.3 Determinism

Determinism refers to the consistency of execution time for each operation.

7.1.4 Resource Efficiency

This includes how efficiently hardware resources such as registers and logic elements are utilized.

7.2 Latency Analysis

In the proposed design, latency is primarily determined by:

- EMA computation
- Signal logic processing
- Data propagation through pipeline stages

Since the system is fully synchronous and pipeline-based, the latency is very low and predictable.

Observation:

- EMA computation: ~1 clock cycle
- Signal generation: ~1 clock cycle

Total effective latency: ≈ 1–2 clock cycles

Given a 100 MHz clock (10 ns period): Total latency ≈ 10–20 ns

This is significantly lower than software-based systems.

7.3 Throughput Analysis

The system processes one input sample per clock cycle.

Throughput:

Throughput = 1 sample / clock cycle

At 100 MHz: 100 million samples per second

This demonstrates high processing capability suitable for real-time systems.

7.4 Deterministic Behavior

Unlike software systems, FPGA-based designs exhibit deterministic behavior.

Key Points:

- No operating system interference
- No context switching
- Fixed execution time

This ensures consistent and predictable performance, which is critical in trading applications.

7.5 FPGA vs Software-Based Systems

A comparison between FPGA and CPU-based systems is presented below:

Table 8: Advantages of FPGA over CPU based system

Parameter	CPU-Based System	FPGA-Based System
Latency	Microsecond	Nanosecond
Execution	Sequential	Parallel
Determinism	Low	High
Throughput	Moderate	High
Power Efficiency	Moderate	High

Analysis:

- CPU systems suffer from software overhead and unpredictable delays
- FPGA systems execute logic directly in hardware, reducing latency
- Parallel processing in FPGA improves performance significantly

7.6 Efficiency of Fixed-Point Arithmetic

The use of Q16.16 fixed-point representation improves system efficiency.

Advantages:

- Faster computation
- Lower hardware complexity
- Reduced power consumption

Trade-off:

- Slight loss in precision compared to floating-point

However, for trading applications, this level of precision is sufficient.

7.7 Parallel Processing Advantage

In this system:

- EMA-14, EMA-50, RSI, and volume calculations run simultaneously
- No dependency between modules

This parallel execution significantly reduces computation time compared to sequential processing.

7.8 Scalability of the System

The modular design allows easy scalability:

- Additional indicators can be added
- Depth of shift registers can be increased
- Multiple stocks can be processed

This makes the system adaptable for more complex applications.

7.9 Limitations of the Current Design

While the system performs efficiently, certain limitations exist:

- Simulation-based validation only
- No real-time market data integration
- Limited complexity of trading strategy
- RSI and volume conditions simplified during testing

.7.10 Summary

This chapter analyzed the performance of the proposed system and demonstrated its advantages over traditional software-based approaches. The results confirm that FPGA-based implementation provides significant improvements in speed, efficiency, and reliability.

CHAPTER 8: CHALLENGES AND LIMITATIONS

During the design and implementation of the FPGA-based stock signal generation system, several practical challenges were encountered. These challenges were related to hardware design constraints, signal synchronization, debugging complexity, and validation of trading logic.

This chapter discusses the major issues faced during the project, along with the approaches used to resolve them. It also highlights the limitations of the current system.

8.1 Challenges Faced During Implementation

8.1.1 Difficulty in Crossover Detection

One of the main challenges was correctly detecting the crossover between fast EMA and slow EMA.

Initially, the system compared current EMA values directly, which resulted in:

- Missing crossover events
- Incorrect or no signal generation

Solution:

A previous-state register was introduced to store the earlier condition. Crossover detection was then implemented by comparing the previous and current states.

8.1.2 Handling Sequential vs Combinational Logic

Another challenge was deciding whether to implement logic in combinational or sequential form.

- Direct combinational comparison led to unstable outputs
- Signals were not synchronized with the clock

Solution:

All critical logic, especially signal generation, was moved into clocked (sequential) blocks using posedge clk.

8.1.3 EMA Initialization Issue

At the beginning of simulation, EMA values were initialized to zero. This caused:

- Incorrect early outputs
- Delay in reaching meaningful values

Solution:

The system was allowed to run for several cycles before considering outputs as valid. The valid signal was used to indicate when EMA values became reliable.

8.1.4 Debugging Signal Not Changing (Always HOLD)

A major issue observed during simulation was that the output remained constant at HOLD, even when price trends changed.

Possible reasons:

- No proper crossover
- Conditions too strict (RSI + volume + EMA together)
- Incorrect signal timing

Solution:

- Simplified signal logic temporarily (focused on EMA crossover)
- Verified waveform step-by-step
- Gradually reintroduced conditions

8.1.5 Testbench Design Complexity

Designing an effective testbench was also challenging.

Initial issues included:

- Incorrect placement of loops
- Multiple initial blocks causing errors
- Improper signal timing

Solution:

- Organized testbench into a single initial block
- Created structured input patterns (uptrend/downtrend)
- Used helper tasks for clarity

8.1.6 Fixed-Point Arithmetic Complexity

Implementing Q16.16 arithmetic required careful handling.

Challenges included:

- Scaling values correctly
- Avoiding overflow
- Maintaining precision

Solution:

- Used 64-bit intermediate variables
- Applied right shifts after multiplication
- Verified values through simulation

8.1.7 Volume Spike Integration Issue

Integrating volume into signal logic created problems:

- Volume spike condition rarely triggered
- Signals were suppressed due to strict conditions

Solution:

- Temporarily bypassed volume condition during debugging
- Later adjusted logic for better integration

8.1.8 Signal Synchronization Issues

Different modules produced outputs at slightly different times.

This caused:

- Mismatch in signal timing

- Incorrect decision making

Solution:

- Used valid signals
- Ensured all modules operated on the same clock
- Synchronized data flow

8.2 Limitations of the System

Despite successful implementation, the system has certain limitations:

8.2.1 Simulation-Based Validation Only

The system has been tested only in simulation environment and not deployed on actual FPGA hardware.

8.2.2 Simplified Trading Strategy

The signal generation logic is based mainly on EMA crossover and does not include advanced trading strategies.

8.2.3 No Real-Time Data Integration

The system uses artificial testbench inputs instead of real stock market data.

8.2.4 Limited Indicators

Only a few indicators (EMA, RSI, Volume) are used. Real trading systems often use more complex combinations.

8.2.5 Precision Limitations

Fixed-point arithmetic introduces minor precision loss compared to floating-point calculations.

8.3 Learning Outcomes

Despite the challenges, the project provided valuable learning:

- Understanding of hardware design using Verilog
- Importance of timing and synchronization
- Practical debugging techniques
- Insight into real-world trading systems

8.4 Summary

This chapter discussed the challenges faced during the design and implementation of the system and the solutions adopted to overcome them. It also highlighted the limitations of the current design.

These insights are important for improving the system in future work and for understanding practical aspects of hardware-based design.

CHAPTER 9: CONCLUSION AND FUTURE WORK

9.1 Conclusion

In this project, a hardware-based stock signal generation system was successfully designed and simulated using FPGA design principles. The system processes stock price and volume data and generates trading signals in the form of BUY, SELL, or HOLD decisions.

The implementation was carried out using Verilog HDL at the Register Transfer Level (RTL), with a modular architecture consisting of key components such as shift registers, EMA calculators, RSI computation unit, volume filter, and signal logic module. Each module was designed to operate synchronously with the system clock, ensuring stable and predictable performance.

The use of Exponential Moving Averages (EMA) enabled effective trend detection, while crossover logic provided a reliable mechanism for identifying potential trading opportunities. The system also incorporated additional factors such as RSI and volume analysis to enhance decision-making, although simplified logic was used during simulation to ensure proper validation.

Simulation results demonstrated that the system correctly identifies trend changes and generates appropriate signals based on EMA crossover conditions. The waveform analysis confirmed that the fast EMA responds quickly to price changes, while the slow EMA provides a smoother representation of the overall trend. The signal generation logic successfully detected crossover events and produced corresponding BUY and SELL outputs.

One of the key achievements of this project is the demonstration of how financial algorithms can be implemented in hardware to achieve low-latency and high-performance processing. Compared to traditional software-based systems, the FPGA-based approach offers deterministic execution, parallel processing, and significantly reduced latency.

Overall, the project successfully meets its objectives by designing and validating a real-time trading signal generator using FPGA methodology. It also provides a strong foundation for further exploration of hardware-accelerated financial systems.

9.2 Future Work

While the current system demonstrates the core concepts of FPGA-based trading signal generation, several enhancements can be made to improve its functionality and practical applicability.

9.2.1 Real-Time Market Data Integration

The system currently uses simulated input data. Future work can involve integrating real-time stock market data through APIs or data feeds, allowing the system to operate in a live environment.

9.2.2 Implementation on Physical FPGA Hardware

The design can be deployed on an actual FPGA board to evaluate real hardware performance, including latency, resource utilization, and power consumption.

9.2.3 Advanced Trading Strategies

More sophisticated strategies can be implemented, such as:

- MACD (Moving Average Convergence Divergence)
- Bollinger Bands
- Machine learning-based prediction models

9.2.4 Multi-Stock Processing

The system can be extended to process multiple stocks simultaneously by duplicating modules and optimizing resource usage.

9.2.5 Improved Signal Accuracy

Additional conditions and filters can be incorporated to reduce false signals and improve decision accuracy.

9.2.6 Optimization of Resource Utilization

Further optimization techniques can be applied to reduce hardware usage and improve efficiency, especially for large-scale implementations.

9.2.7 Integration with Trading Platforms

The system can be connected to actual trading platforms to automate order execution based on generated signals.

9.3 Final Remarks

This project highlights the potential of FPGA-based systems in real-time data processing applications such as financial trading. It demonstrates that hardware acceleration can significantly improve performance and enable faster decision-making compared to traditional approaches.

The knowledge gained through this project can be applied to a wide range of applications beyond trading, including signal processing, embedded systems, and real-time analytics.

REFERENCES

[1] C. Leber, B. Geib, and H. Litz, "High-Frequency Trading Acceleration Using FPGAs," in Proc. IEEE, 2011.

[2] Y. C. Kao, H. A. Chen, and H. P. Ma, "An FPGA-Based High-Frequency Trading System for 10 Gigabit Ethernet with a Latency of 433 ns," in Proc. IEEE VLSI-DAT, 2022.

[3] Q. Tang et al., "A Scalable Architecture for Low-Latency Market Data Processing on FPGA," in Proc. IEEE ISCC, 2016.

[4] M. Dvorak and J. Korenek, "Low Latency Book Handling in FPGA for High-Frequency Trading," in Proc. IEEE, 2014.

[5] H. Jia et al., "A Domain-Specific Accelerator for Ultra-Low Latency Market Data Distribution System," in Proc. IEEE FCCM, 2022.

[6] D. Gupta et al., "FPGA for High-Frequency Trading: Reducing Latency in Financial Systems," in Proc. IEEE ICACRS, 2024.

[7] A. Boutros et al., "Build Fast, Trade Fast: FPGA-Based High-Frequency Trading Using High-Level Synthesis," 2018.

[8] S. Puranik et al., "Acceleration of Trading System Back End with FPGAs Using High-Level Synthesis," Electronics, vol. 12, no. 3, 2023.

[9] A. Ali et al., "A Framework for High-Frequency Trading Algorithms on FPGA Using Zynq SoC," 2024.

[10] Jane Street Capital, "Quantitative Trading Firm Overview."

[11] Citadel Securities, "Global Market Maker and High-Frequency Trading Firm."

[12] Jump Trading, "High-Speed Algorithmic Trading Firm."

[13] Hudson River Trading, "Algorithmic Trading Company."

[14] Optiver, "Electronic Market Making Firm."

[15] IMC Trading, "Global Trading Firm."

[16] Virtu Financial, "High-Frequency Trading Firm."

[17] Flow Traders, "ETF Market Making Firm."

[18] Tower Research Capital, "Quantitative Trading Firm."

[19] DRW Trading Group, "Proprietary Trading Firm."

[20] Xilinx Inc., "Zynq-7000 SoC Technical Reference Manual," 2020.

[21] Xilinx Inc., "Vivado Design Suite User Guide," 2022.

[22] Xilinx Inc., "Zynq-7000 SoC Data Sheet," 2021.

[23] ARM Ltd., "AMBA AXI Protocol Specification," 2019.

[24] Xilinx Inc., "ZedBoard (Zynq FPGA) User Guide," 2021.

[25] J. J. Murphy, "Technical Analysis of the Financial Markets," New York Institute of Finance, 1999.

[26] Investopedia, "Relative Strength Index (RSI)."

[27] Investopedia, "Exponential Moving Average (EMA)."

APPENDIX

Appendix A: Exponential Moving Average (EMA) Module

```
module ema #(
parameter DATA_WIDTH = 32,
parameter ALPHA = 8738,
parameter ONE_M_ALPHA = 56798
)(
input wire clk, rst, en,
input wire [DATA_WIDTH-1:0] price_in,
output reg [DATA_WIDTH-1:0] ema_out,
output reg valid
);

reg seeded;

wire [63:0] term1 = ({32'd0, price_in} * ALPHA) >> 16;
wire [63:0] term2 = ({32'd0, ema_out}  * ONE_M_ALPHA) >> 16;

always @(posedge clk) begin
   if (rst) begin
      ema_out <= 0;
      valid   <= 0;
      seeded  <= 0;
   end
   else if (en) begin
      if (!seeded) begin
         ema_out <= price_in;
         seeded  <= 1;
         valid   <= 0;
      end
      else begin
         ema_out <= term1[DATA_WIDTH-1:0] + term2[DATA_WIDTH-1:0];
         valid   <= 1;
      end
   end
   else valid <= 0;
end


endmodule
```

Appendix B: Relative Strength Index (RSI) Module

```
module rsi_calc #(
parameter DATA_WIDTH = 32,
parameter PERIOD = 14,
parameter SMOOTH_OLD = 60855,
parameter SMOOTH_NEW = 4681
)(
input wire clk, rst, en,
input wire [DATA_WIDTH-1:0] price_in,
output reg [DATA_WIDTH-1:0] rsi_out,
output reg valid
);

reg [DATA_WIDTH-1:0] prev_price, avg_gain, avg_loss;
reg initialized;

wire signed [DATA_WIDTH:0] delta =
   $signed({1'b0, price_in}) - $signed({1'b0, prev_price});

wire [DATA_WIDTH-1:0] gain = (delta > 0) ? delta[DATA_WIDTH-1:0] : 0;
wire [DATA_WIDTH-1:0] loss = (delta < 0) ? (-delta[DATA_WIDTH-1:0]) : 0;

wire [63:0] gain_prod = (avg_gain * SMOOTH_OLD) + (gain * SMOOTH_NEW);
wire [63:0] loss_prod = (avg_loss * SMOOTH_OLD) + (loss * SMOOTH_NEW);

wire [DATA_WIDTH-1:0] new_avg_gain = gain_prod[47:16];
wire [DATA_WIDTH-1:0] new_avg_loss = loss_prod[47:16];

wire [63:0] num   = (64'd100 * {32'd0, new_avg_gain}) << 16;
wire [63:0] denom = {32'd0, new_avg_gain} + {32'd0, new_avg_loss};

wire [63:0] rsi_64 = (denom == 0) ? (64'd50 << 16) : (num / denom);

always @(posedge clk) begin
   if (rst) begin
      prev_price <= 0; avg_gain <= 0; avg_loss <= 0;
      initialized <= 0; rsi_out <= 32'd50 << 16; valid <= 0;
   end
   else if (en) begin
      if (!initialized) begin
         prev_price <= price_in;
         initialized <= 1;
         valid <= 0;
```

```
        end
        else begin
            avg_gain <= new_avg_gain;
            avg_loss <= new_avg_loss;
            prev_price <= price_in;
            rsi_out <= rsi_64[DATA_WIDTH-1:0];
            valid <= 1;
        end
    end
    else valid <= 0;
end


endmodule
```

Appendix C: Simple Moving Average (SMA) Module

```
module sma #(
parameter DATA_WIDTH = 32,
parameter DEPTH = 14,
parameter RECIP = 4681
)(
input wire clk, rst, en,
input wire [DATA_WIDTH-1:0] new_sample, old_sample,
output reg [DATA_WIDTH-1:0] sma_out,
output reg valid
);

reg [63:0] running_sum;

always @(posedge clk) begin
    if (rst) begin
        running_sum <= 0; sma_out <= 0; valid <= 0;
    end
    else if (en) begin
        running_sum <= running_sum + new_sample - old_sample;
        sma_out <= (running_sum * RECIP) >> 16;
        valid <= 1;
    end
    else valid <= 0;
end


endmodule
```

Appendix D: Volume Filter Module

```
module volume_filter #(
parameter DATA_WIDTH = 32,
parameter DEPTH = 20,
parameter RECIP_20 = 3277,
parameter THRESH_Q = 98304
)(
input wire clk, rst, en,
input wire [DATA_WIDTH-1:0] vol_in,
output wire [DATA_WIDTH-1:0] avg_vol_out,
output reg vol_spike
);

reg [DATA_WIDTH-1:0] vol_mem [0:DEPTH-1];
reg [63:0] vol_sum;
integer i;

wire [DATA_WIDTH-1:0] avg_vol = (vol_sum * RECIP_20) >> 16;
assign avg_vol_out = avg_vol;

wire [63:0] threshold_64 = (avg_vol * THRESH_Q) >> 16;
wire [DATA_WIDTH-1:0] threshold = threshold_64[DATA_WIDTH-1:0];

always @(posedge clk) begin
   if (rst) begin
      for (i=0;i<DEPTH;i=i+1) vol_mem[i]<=0;
      vol_sum<=0; vol_spike<=0;
   end
   else if (en) begin
      vol_sum <= vol_sum + vol_in - vol_mem[DEPTH-1];
      for (i=DEPTH-1;i>0;i=i-1) vol_mem[i]<=vol_mem[i-1];
      vol_mem[0]<=vol_in;
      vol_spike <= (vol_in > threshold);
   end
end


endmodule
```

Appendix E: Signal Generation Logic

```
module signal_logic #(
   parameter DATA_WIDTH = 32,
   parameter RSI_80     = 32'd5242880,   // 80 * 65536  (was RSI_70 = 70*65536)
  parameter RSI_10 = 32'd655360    // 20 * 65536  (was RSI_30 = 30*65536)
)(
   input  wire                clk,
   input  wire                rst,

   input  wire [DATA_WIDTH-1:0] ma_fast,
   input  wire [DATA_WIDTH-1:0] ma_slow,
   input  wire                ma_valid,

   input  wire [DATA_WIDTH-1:0] rsi,
   input  wire                rsi_valid,

   input  wire                vol_spike,

   output reg  [1:0]            signal_out,
   output reg                 signal_valid
);

   reg  initialized;
   reg  fast_above_d;

   wire fast_above   = (ma_fast > ma_slow);
   wire crossover_up = fast_above  && !fast_above_d;
   wire crossover_dn = !fast_above && fast_above_d;

   // STAGE 2: relaxed RSI guards (80/20 instead of 70/30)
   // At step 130, RSI~78 - fits under 80, so BUY fires.
   // Once confirmed, you can tighten back to 70/30 with real data.
   wire rsi_ok_buy  = (rsi < RSI_80);   // not overbought (< 80)
   wire rsi_ok_sell = (rsi > RSI_10);   // not oversold  (> 20)

   // STAGE 3 (re-add when BUY/SELL confirmed firing):
   // wire rsi_ok_buy  = (rsi < RSI_80) && vol_spike;

   always @(posedge clk) begin
      if (rst) begin
         signal_out   <= 2'b00;
         signal_valid <= 1'b0;
         fast_above_d <= 1'b0;
         initialized  <= 1'b0;
      end
      else if (ma_valid) begin
         if (!initialized) begin
            fast_above_d <= fast_above;
            initialized  <= 1'b1;
```

```
            signal_out   <= 2'b00;
            signal_valid <= 1'b0;
        end
        else begin
            if (crossover_up && rsi_ok_buy)
                signal_out <= 2'b01;   // BUY

            else if (crossover_dn && rsi_ok_sell)
                signal_out <= 2'b10;   // SELL

            else
                signal_out <= 2'b00;   // HOLD

            signal_valid <= 1'b1;
            fast_above_d <= fast_above;
        end
    end
    else begin
        signal_valid <= 1'b0;
    end
  end

endmodule
```

Appendix F: Testbench

This testbench simulates different market trends to verify system behavior.

```
`timescale 1ns / 1ps

module tb_stock_signal;

  parameter DATA_WIDTH = 32;
  parameter CLK_PERIOD = 10;

  reg                  clk, rst, en;
  reg  [DATA_WIDTH-1:0] price_in, volume_in;

  wire [1:0]  signal_out;
  wire        signal_valid;
  wire [2:0]  led;

  stock_signal_top #(.DATA_WIDTH(DATA_WIDTH)) dut (
    .clk(clk), .rst(rst), .en(en),
    .price_in(price_in), .volume_in(volume_in),
    .signal_out(signal_out), .signal_valid(signal_valid),
    .led(led)
  );

  initial clk = 0;
  always #(CLK_PERIOD/2) clk = ~clk;
```

```
// Send one sample
task send_sample;
    input integer price, volume;
    begin
        price_in  = price  << 16;
        volume_in = volume << 16;
        en = 1; @(posedge clk); #1;
        en = 0; @(posedge clk); #1;
    end
endtask

// Print every step
task print_signal;
    input integer step, price;
    begin
        $write("Step %3d  px=%3d  EMA14=%6.1f  EMA50=%6.1f  RSI=%5.1f  |  ",
            step, price,
            dut.ema14_out / 65536.0,
            dut.ema50_out / 65536.0,
            dut.rsi_out   / 65536.0);
        case (signal_out)
            2'b01: $display(">>> BUY  <<<");
            2'b10: $display(">>> SELL <<<");
            2'b00: $display("    hold");
            default: $display("????");
        endcase
    end
endtask

integer k, pv;

initial begin
    rst=1; en=0; price_in=0; volume_in=0;
    repeat(5) @(posedge clk);
    rst=0; @(posedge clk);

    $display("=== SIMULATION START ===");
    $display("Watch EMA14 and EMA50 - BUY fires when EMA14 crosses above EMA50");
    $display("");

    // Phase 1: downtrend 200->101 (100 steps)
    $display("--- PHASE 1: Downtrend 200->101 ---");
    for (k=0; k<100; k=k+1) begin
        pv = 200 - k;
        send_sample(pv, 1000000);
        print_signal(k, pv);
    end

    // Phase 2: uptrend 50->248 step=2 (100 steps)
    $display("");
```

```
        $display("--- PHASE 2: Uptrend 50->248 (step +2) ---");
        for (k=0; k<100; k=k+1) begin
            pv = 50 + k*2;
            send_sample(pv, 2000000);
            print_signal(k+100, pv);
        end

        // Phase 3: downtrend 250->171 (80 steps)
        $display("");
        $display("--- PHASE 3: Downtrend 250->171 ---");
       // NEW - steeper drop, step=-2
for (k=0; k<100; k=k+1) begin
    pv = 250 - k*2;
    if (pv < 20) pv = 20;   // floor so price doesn't go negative
    send_sample(pv, 1000000);
    print_signal(k+200, pv);
end

        $display("=== DONE ===");
        $finish;
    end

    initial begin
        $dumpfile("stock_signal.vcd");
        $dumpvars(0, tb_stock_signal);
    end

endmodule
```